Multi – Level Linked List

–Insert At End

–Space Complexity and Algorithm

Program:

```
Node *createNode(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Memory allocation failed." << endl;
        exit(1);
    }
    newNode->data = data;
    newNode->next = nullptr;
    newNode->child = nullptr;
    return newNode;
}
// --- 2. Insert at End (Main Level) ---
void insertAtEnd()
{
    int data;
    cout << "Enter data: ";
    cin >> data;

    Node *newNode = createNode(data);

    if (head == nullptr)
    {
        head = newNode;
    }
    else
    {
        Node *temp = head;
        while (temp->next != nullptr)
        {
```

```
            temp = temp->next;
        }
        temp->next = newNode;
    }
    cout << "Inserted " << data << " at end." << endl;
}
int main()
{
    int choice;
    while (true)
    {

       cout << "2. Insert at End (Main)" << endl;

        switch (choice)
       {
        case 2:
            insertAtEnd();
            break;
       default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

Space Complexity Analysis

1. *Input Space Complexity*:

A. (*Parameter − Only Definition*):

```
void insertAtEnd(){…}
```

No parameters are passed.

So under the parameter − only definition:

$$O(1)$$

because the function input size through parameters is constant.

B. Full Data Definition[*Holisitic Approach*]

Now considering the entire linked list data stored in memory.

Each insertion creates one node:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

Each node contains[*In CreateNode Function*]:

```
    int data;
    struct Node *next;  // Points to the right (Same Level)
    struct Node *child; // Points down (Sub Level)
```

Each insertion increases memory linearly.

So after storing `n` nodes:

$$O(n)$$

This is the total data structure space.

C. Auxiliary Space Complexity

Extra temporary memory used by the algorithm during execution,

excluding the final input/output data structure.

It measures:

- ***Local variables***
- ***Temporary pointers***
- ***Function call stack***
- ***Recursive stack frames***
- ***Temporary arrays/buffers***

But usually excludes:

- ***Original input data***
- ***Final permanent data structure(Full Data Definition or Holistic Approach)***

$int\ data; \rightarrow O(1) space$

***Inside function of InsertAtEnd*:**

```
int data;
Node *newNode;
```

Also in createNode(int data) function:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

***Here , both the newNode contains dynamically memory addresses, where as , dynamically allocated memory* :**

```
(Node *)malloc(sizeof(Node));
```

***itself is input space. But not*:**

```
Node *newNode;
```

*Hence Node * newNode; → O(1) auxiliary space.*

Also,

```
      Node *temp = head;
```

***Again*:**

- ***Single pointer***
- ***Constant size***
- ***Reused during traversal***

***Even though it moves through many nodes*:**

```
       while (temp->next != nullptr)
        {
            temp = temp->next;
        }
```

runs many times.

But:

- ***Time increases***
- ***Space does NOT increase***

Because no new memory is created each iteration.

Only the same pointer changes value.

Hence it takes only , $O(1)$constant space complexity.

Total Auxiliary Memory Used

Variable	***Type***	***Space***
data	***int***	$O(1)$
newNode	***pointer***	$O(1)$
temp	***pointer***	$O(1)$

Total:

$$O(1) + O(1) + O(1) = O(1)$$

Final Auxiliary Space Complexity is : $O(1)$.

because the extra temporary memory remains constant regardless of list size.

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\underbrace{Space}_{} = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(1)}_{\text{Input}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\underbrace{Space}_{} = \underbrace{O(1)}_{\text{Auxiliary}} + \underbrace{O(n)}_{\text{Input}} = O(n).$$

1) If Asked "Total Space Complexity" then natual answer is
: O(n) [*Full Data + Auxiliary*]

2) *If not asked total space complexity then*:

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

In most academic DSA contexts and coding interviews, when someone simply asks:

“What is the space complexity?”

they usually mean:

$$\boxed{\text{Auxiliary Space Complexity}}$$

unless they explicitly say:

- total space complexity
- overall memory usage
- data structure space
- input space
- storage complexity

So for your insertion function, the expected answer is usually:

$$\boxed{O(1)}$$

because only constant extra working memory is used.

Algorithm:

MultiINSEnd():

1. *Declare an integer type variable data.*
2. *Read data.*
3. *SET NEWPTR* := CALL *CreateNode*(*data*)

Translating CreateNode(*data*)

(((***Node*** ∗)***malloc***(***sizeof***(***Node***)) ***is availability of space for insertion of Data, hence, lets name it AVAIL.***))

CreateNode(*data*):

1. *Allocate New Node*

   SET NewTempPTR := *AVAIL.*

2. *Memory Allocated* ?:

   If NewTempPTR = NULL, then:
   Write: " Error: Memory allocation failed."
   Return and EXIT.
   [*End of If structure.*]

3. *Copy Data*:

   Set INFO[*NewTempPTR*] := *data*

4. *Set Next section of Allocated Memory*:

   Set Link[*NewTempPTR*] := *NULL*

5. *Set Child section of Allocated Memory*:

$$Set\ childLink[NewTempPTR]\ := NULL$$

6. *Return Stored Address in NewTempPTR*

$$return\ NewTempPTR$$

---

4. [***If LIST is EMPTY***?]

If START = NULL , Then:

$$SET\ \ START := NEWPTR$$

[***If List Is Not Empty***?]

ELSE :

$$SET\ TEMPPTR := START\ \begin{bmatrix} Temporary\ Pointer\ points\ to \\ First\ Node \end{bmatrix}$$

$$REPEAT\ \ WHILE\ LINK[TEMPPTR]\ \neq\ NULL : \begin{bmatrix} List \\ Traversal \end{bmatrix}$$

$$SET\quad TEMPPTR := LINK[TEMPPTR]$$

$$SET\ \ LINK[TEMPPTR] := NEWPTR. \begin{bmatrix} Link\ Last\ Node\ to\ the \\ Newly\ created\ Node \end{bmatrix}$$

Write : ``Inserted``, *data*, ``*at end*`` .

5. ***END OF ALGORITHM***